

Missouri University of Science and Technology

Intro to Data Science
CS 3402 (Spring 2026)

Lecture 6 Descriptive Statistics II & Derivatives for DL/ML

Xiaowei Yu, Ph.D.

Department of Computer Science

Descriptive Statistics

Discrete Distribution

Probability Mass Function (PMF) describes probability distribution of a **discrete random variable (RV) X**

Formally, for a discrete random variable X,

$$p_X(x) = P(X = x)$$

1. Non-negativity; 2. Total probability = 1

What is $P(X \leq x)$

Result of $P(X \leq 4)$?

e.g., a variable X that takes **countable values** like [1, 2, 3, 4, 5] ....

$$p_X(2) = P(X = 2)$$

x	1	2	3	4	5
P(X=x)	1/5	1/5	1/5	1/5	1/5

Descriptive Statistics

Continuous Distribution

Probability Density Function (PDF) describes probability distribution of a **continuous random variable (RV) X**

Formally, for a random continuous variable X, a **Cumulative Distribution Function** (CDF) is defined as:

$$F_X(x) = \int_{\infty}^{x} f_X(t)\,d_t$$

CDF ← $F_X(x)$; $f_X(t)$ → PDF

For continuous variable, $P(X = x)$?

Concept	PMF	PDF
Used for	**Discrete** RVs	**Continuous** RVs
Probability	P(X=x)	$P(a \leq X \leq b) = \int_a^b f_X(t)\,d_t$
Values sum/integrate to	1 (sum)	1 (integral)

Descriptive Statistics

Cumulative Distribution Function (CDF)

For a **random variable** X, CDF is defined as

$F_X(x) = P(X \leq x)$ ← Here the random variable can be either continuous or discrete variable!

$F_X(x) = \int_{\infty}^{x} f_X(t)dt$ ← valid only when X is continuous!

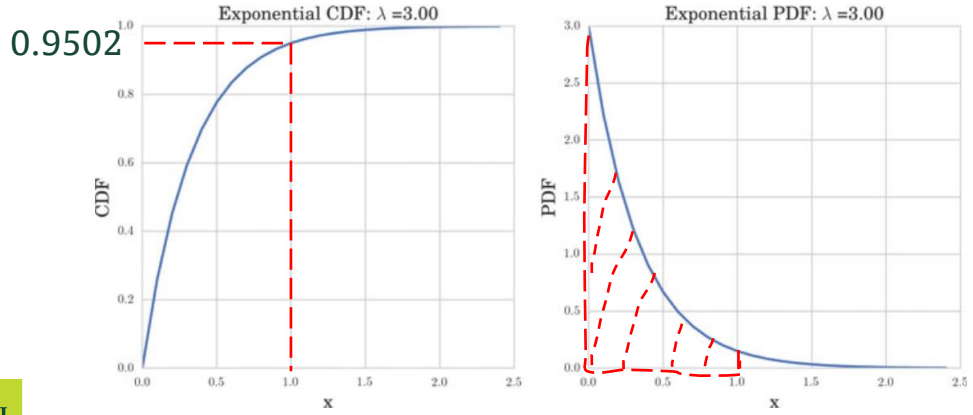
The probability that X is less than or equal to x.

Continuous Distribution

Exponential distribution

$$\text{PDF}(x) = \lambda e^{-\lambda x} \longrightarrow \text{CDF}(x) = \int_{\infty}^{x} PDF(t)\,d_t \longrightarrow \text{CDF}(x) = 1 - \lambda e^{-\lambda x}$$

The parameter λ defines the shape of the distribution.



How to compute $P(x \leq 1)$?

How to compute $P(0.5 \leq x \leq 1)$?

$P(0.5 \leq x \leq 1) = P(x \leq 1) - P(x \leq 0.5)$

Continuous Distribution

The normal distribution/Gaussian distribution

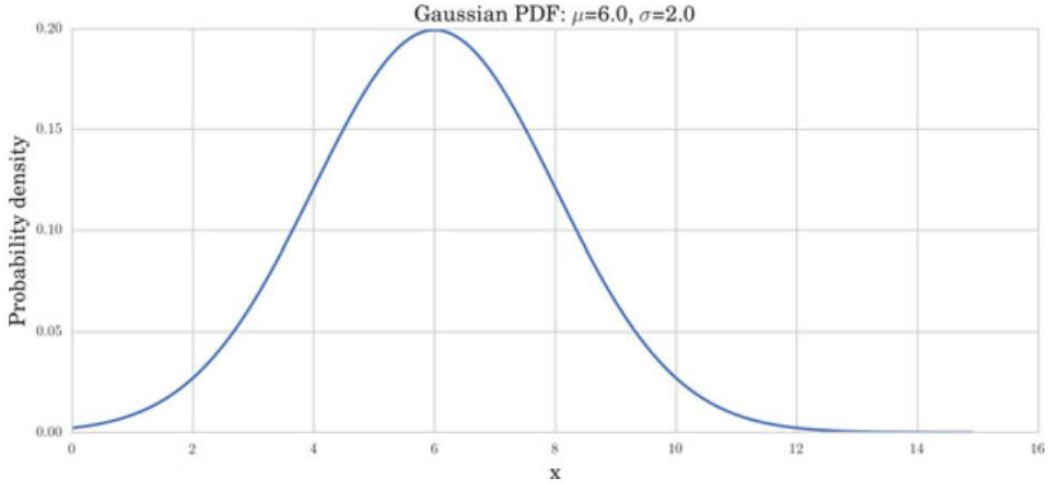
the most common distribution

$$\text{PDF}(x) = \frac{1}{2\pi\sqrt{\sigma^2}} e^{-\frac{(x-\mu)^x}{2\sigma^2}}$$

What is (μ)

What is σ^2

The parameter σ defines the shape of the distribution.



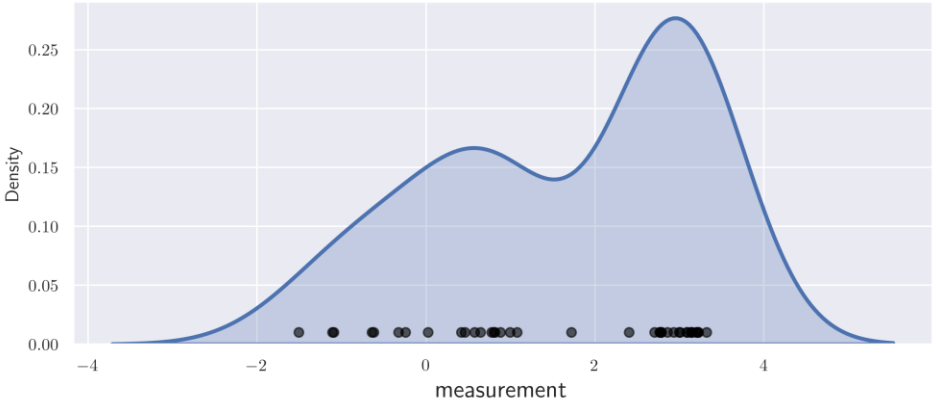
real phenomena with a normal distribution:

- The size of living tissue (length, height, weight)
- Different physiological measurements (e.g., blood pressure), etc.

Kernel Density

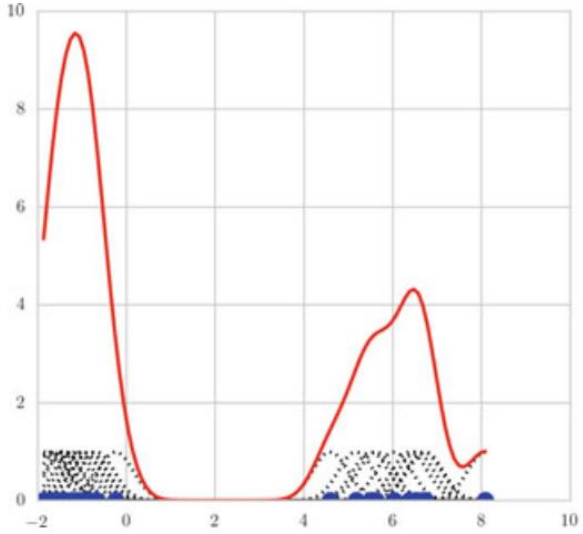
- In many real problems, we do not know the distribution of data.
- We only have finite samples.

Given only data points, how do we estimate the underlying PDF?



Random data shown in black, kernel shown in blue

Approximate unknown underlying PDF using a Gaussian kernel



Random data shown in blue, kernel shown in black, summed function in red

Sample and Estimated Metrics

Mean $$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i .$$

Let's assume that we get some random samples from a normal distribution $\text{PDF}(x) = \frac{1}{2\pi\sqrt{\sigma^2}} e^{-\frac{(x-\mu)^x}{2\sigma^2}}$

$$\{0.33, -1.76, 2.34, 0.56, 0.89\}$$

Can we guess the mean μ of the distribution?

Yes, we can use sample mean $\bar{x}$ to approximate the mean μ. This process is called *estimation*, and the sample mean is called an *estimator*.

Sample and Estimated Metrics

Mean $$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i .$$

Let's assume that we get some random samples from a normal distribution $\text{PDF}(x) = \frac{1}{2\pi\sqrt{\sigma^2}} e^{-\frac{(x-\mu)^x}{2\sigma^2}}$

$$\{0.33, -1.76, 2.34, 0.56, 0.89, -460\}$$

The last value seems to be an outlier compared to the rest of the sample.

How to avoid this effect?

Sample and Estimated Metrics

Mean $$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i .$$

$\text{PDF}(x) = \frac{1}{2\pi\sqrt{\sigma^2}} e^{-\frac{(x-\mu)^x}{2\sigma^2}}$

Data samples {0.33, -1.76, 2.34, 0.56, 0.89}

How to measure how good of the estimator? (Mean squared error)

$\text{MSE} = \frac{1}{n}\sum(\bar{x} - \mu)^2$

Sample and Estimated Metrics

Variance σ^2 $$\sigma^2 = \frac{1}{n}\sum_i (x_i - \mu)^2.$$

We can use the sample variance as an estimator.

Data samples {0.33, -1.76, 2.34, 0.56, 0.89}

Let us use the sample variance $\bar{\sigma}^2$ as an estimator of the variance σ^2.

$$\bar{\sigma}^2 = \frac{1}{n}\sum (x_i - \bar{x})^2$$

Sample and Estimated Metrics

Standard Score

Data that come in the same units can belong to different distributions.

Normalize them to standard scores.

Standard scores:

$$z_i = \frac{(x_i - \mu)}{\sigma}$$

Very popular operation in vision data preprocessing for deep learning models!

Covariance

Two variables X and Y share the same tendency, we speak about covariance.

$$Cov(X,Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)$$

n is the length of both sets.

Correlation and the Pearson's Correlation

Definition of Pearson's correlation

$$\rho = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

We have $Cov(X,Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)$

Substitute covariance into Pearson's correlation

$$\rho = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu_x)(y_i-\mu_y)}{\sigma_X \sigma_Y} = \frac{1}{n}\sum_{i=1}^{n}\frac{x_i-\mu_x}{\sigma_X}\frac{y_i-\mu_y}{\sigma_Y}$$

Standard score of variable X

Standard score of variable Y

Correlation and the Pearson's Correlation

Definition of Pearson's correlation

$$\rho = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

We have $Cov(X,Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)$

Substitute covariance into Pearson's correlation

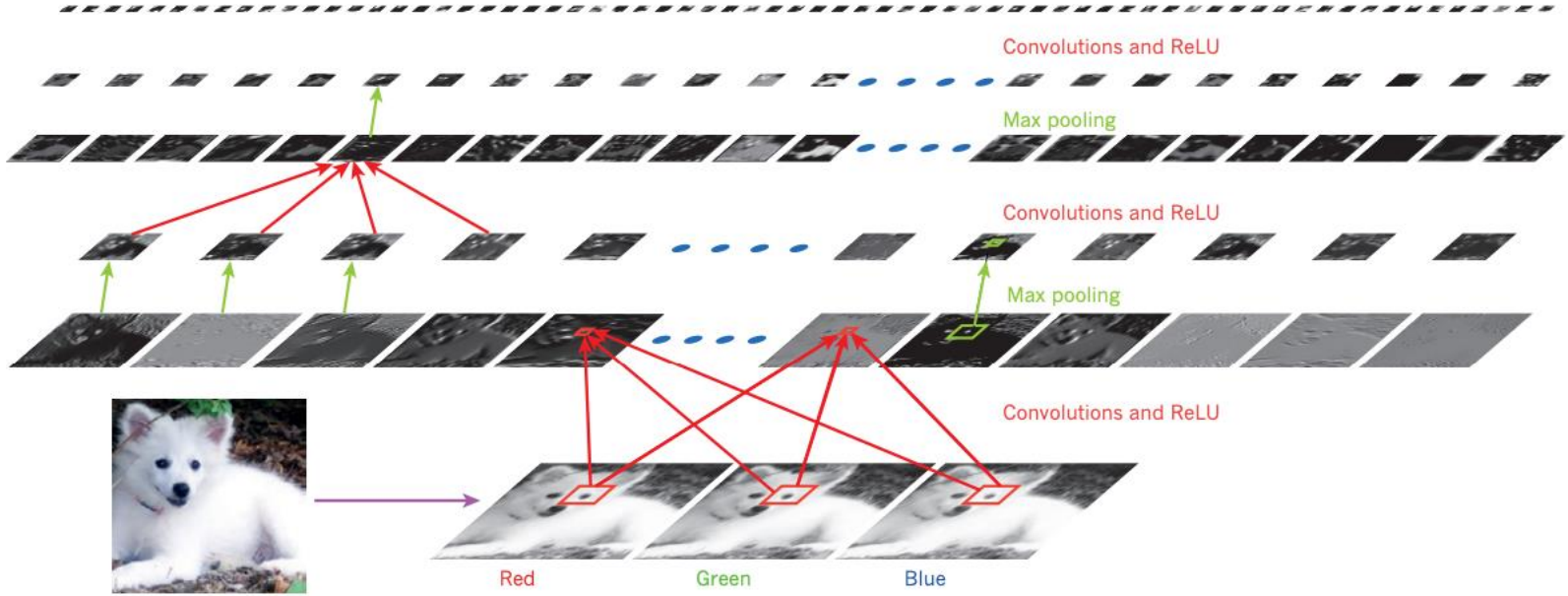
$$\rho = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu_x)(y_i-\mu_y)}{\sigma_X \sigma_Y} = \frac{1}{n}\sum_{i=1}^{n}\frac{x_i-\mu_x}{\sigma_X}\frac{y_i-\mu_y}{\sigma_Y}$$

Standard score of variable X

Standard score of variable Y

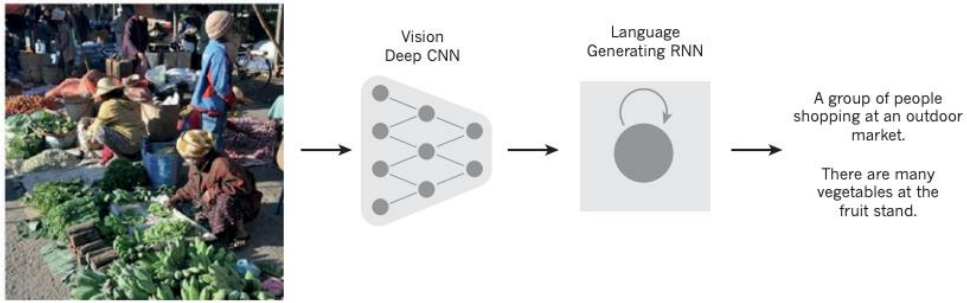
Deep Learning Models

Deep learning enables multiple processing layers (AI models) to learn representations of data with multiple levels of abstraction.



Deep Learning Models

Captions generated by a language model (RNN) taking, as extra input, the representation extracted by a vision model(CNN) from a test image, with the RNN trained to ‘translate’ high-level representations of images into captions (top)




A woman is throwing a **frisbee** in a park.


A **dog** is standing on a hardwood floor.



A **stop** sign is on a road with a mountain in the background

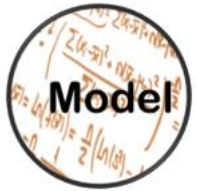
Visualizing the association between regions of interest (ROIs) in images and English words in deep learning models


A little **girl** sitting on a bed with a teddy bear.


A group of **people** sitting on a boat in the water.


A giraffe standing in a forest with **trees** in the background.

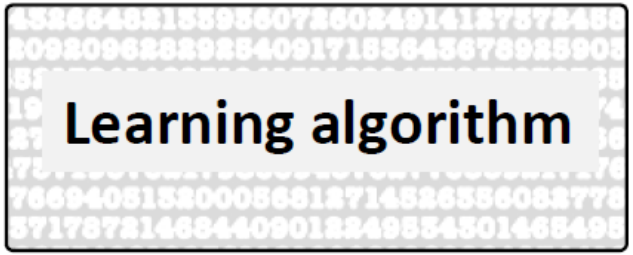
Derivatives in Deep Learning/Machine Learning

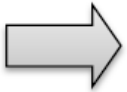


Training data

$\{(X_i, Y_i)\}_{i=1}^{n}$





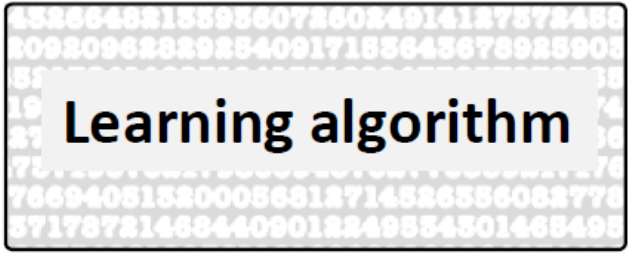


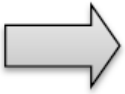
Prediction rule

$\widehat{f}_n$

Testing data

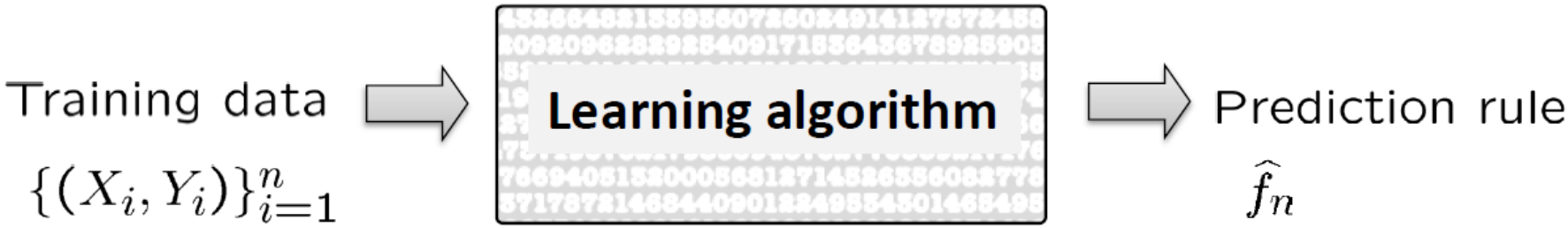






Predicted results

Derivatives in Deep Learning/Machine Learning



E.g.,

Images → Artificial Neural Networks → Predicted Labels

Derivatives in Deep Learning/Machine Learning

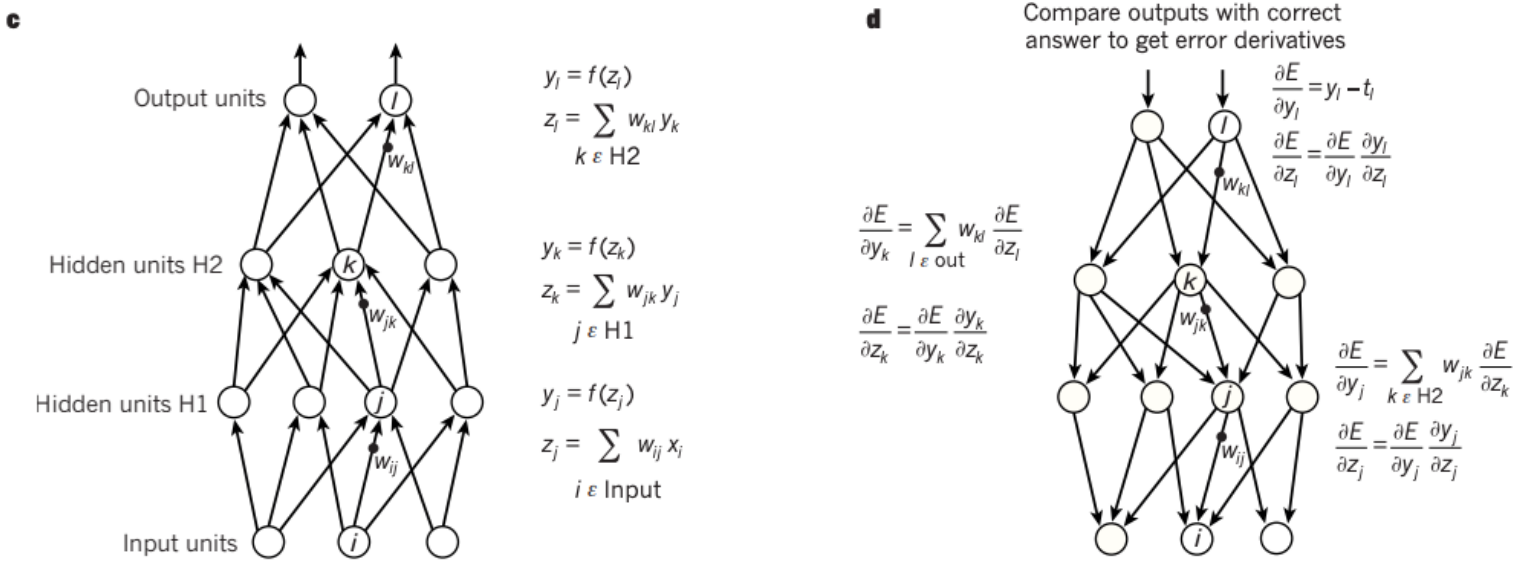
- Goal of deep learning/machine learning: **make predictions with small error**
- We define a **loss function** to measure error
- Derivatives tell us **how to change parameters to reduce loss**

Learning = minimizing loss

Loss depends on model parameters (weights)

$$\mathcal{L}(w) = (y - \hat{y})^2$$

Derivatives in Deep Learning/Machine Learning



Forward pass

$\mathcal{L}(w) = (y - \hat{y})^2$

Back propagation

Derivatives in Deep Learning/Machine Learning

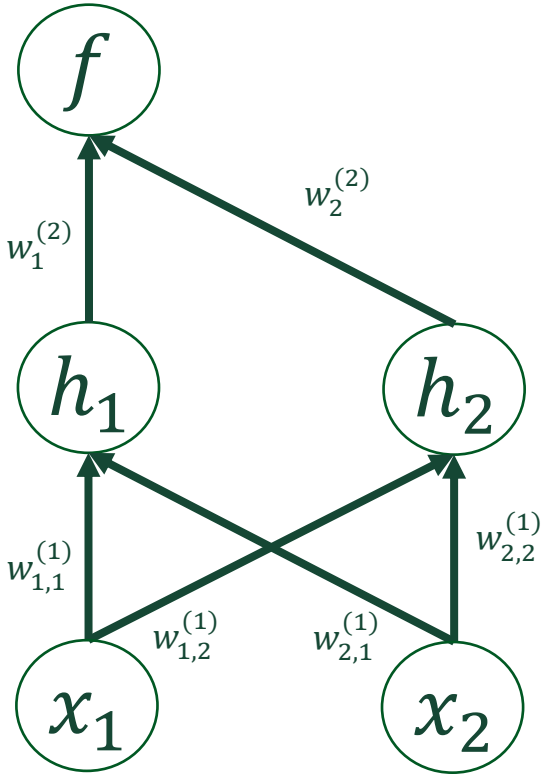
Flow diagrams

$$h_1 = g^{(1)}\left(b_1^{(1)} + w_{1,1}^{(1)} x_1 + w_{1,2}^{(1)} x_2\right)$$

$$h_2 = g^{(1)}\left(b_2^{(1)} + w_{2,1}^{(1)} x_1 + w_{2,2}^{(1)} x_2\right)$$

$$f = g^{(2)}\left(b_1^{(2)} + w_1^{(2)} h_1 + w_2^{(2)} h_2\right)$$

nonlinearities $g^{(l)}$, and some weights $w_{j,k}^{(l)}$ and biases $b_j^{(l)}$

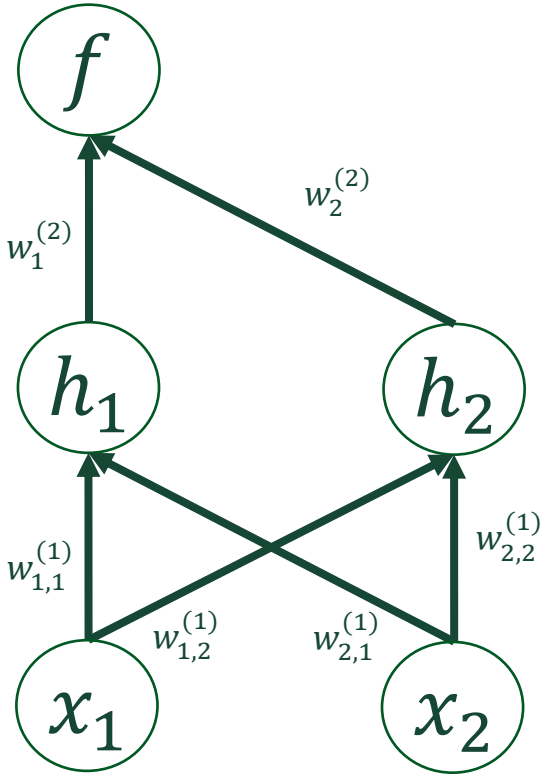


Derivatives in Deep Learning/Machine Learning

Flow diagrams

The important piece of information shown in a flow diagram is the order of computation. This flow diagram shows that, given x_1 and x_2,

- First, you calculate h_1 and h_2,
- then you can calculate f.



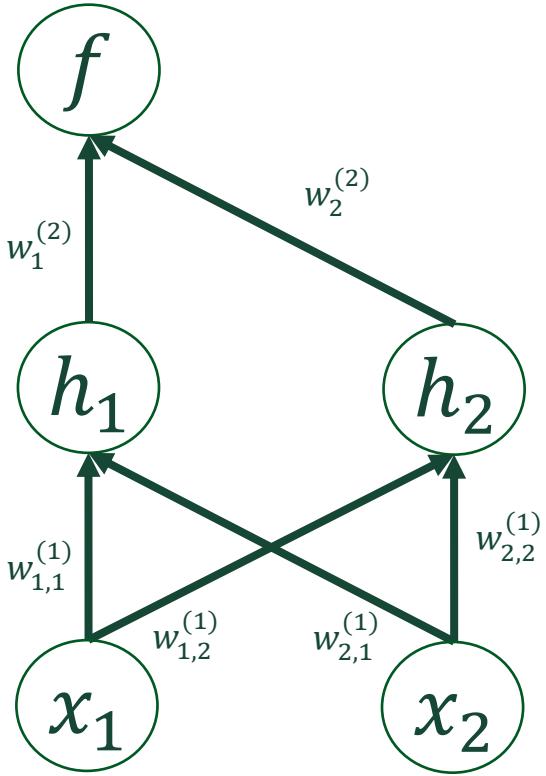
Derivatives in Deep Learning/Machine Learning

Gradient descent: basic idea

- Suppose we have a training token x.
- Its target label is y.
- The neural net produces output $f(x)$, which is not y.
- The difference between y and $f(x)$ is summarized by some loss function, $\mathcal{L}(y, f(x))$.
- The output of the neural net is determined by some parameters, $w_{j,k}^{(l)}$.
- Then we can improve the network by setting:

$$w_{j,k}^{(l)} \leftarrow w_{j,k}^{(l)} - \eta \frac{d\mathcal{L}}{dw_{j,k}^{(l)}}$$

η is the learning rate, always larger than 0



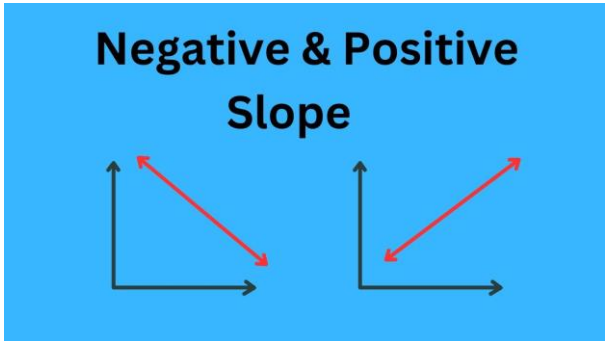
Derivatives in Deep Learning/Machine Learning

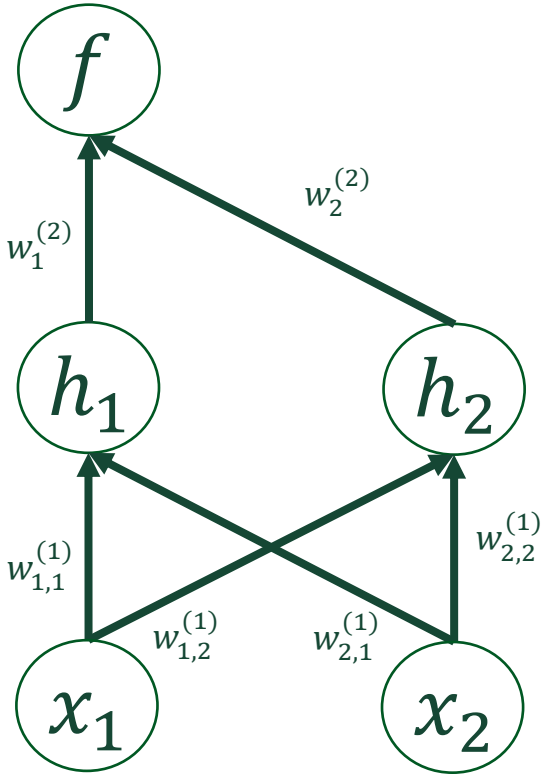
Gradient descent: basic idea

$$\frac{d\mathcal{L}}{dw_{j,k}^{(l)}}$$

- Positive slope → increasing w increases loss
- Negative slope → increasing w decreases loss

slope tells us which way is downhill





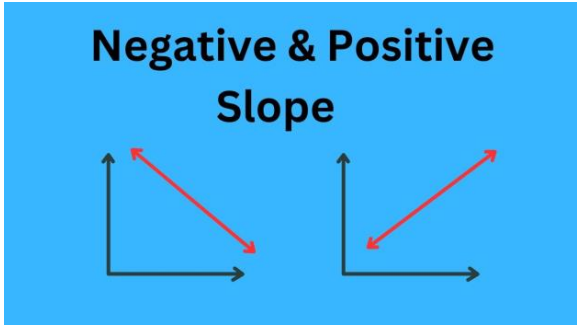
Derivatives in Deep Learning/Machine Learning

Gradient descent: basic idea

$$\frac{d\mathcal{L}}{dw_{j,k}^{(l)}}$$

- Positive slope → increasing w increases loss
- Negative slope → increasing w decreases loss

slope tells us which way is downhill



$$w_{j,k}^{(l)} \leftarrow w_{j,k}^{(l)} - \eta \frac{d\mathcal{L}}{dw_{j,k}^{(l)}}$$

Partial Derivative Examples

$f(x_1, x_2) = 2x_1 + 3x_2 + 4\,x_1\,x_2$

How to calculate the partial derivative for x1 and x2?

$$\frac{\partial \mathcal{L}}{\partial x_1}$$

$$\frac{\partial \mathcal{L}}{\partial x_2}$$

Toy Example of Gradient Descent in ML Models

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

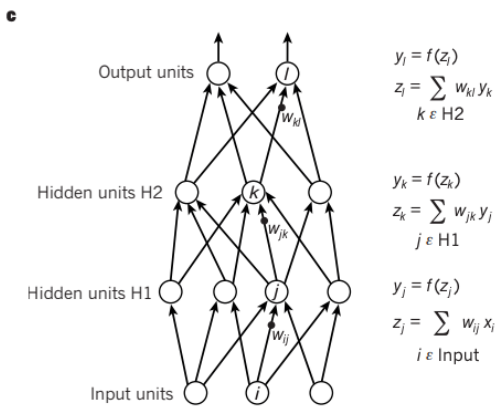
Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate: 0.1

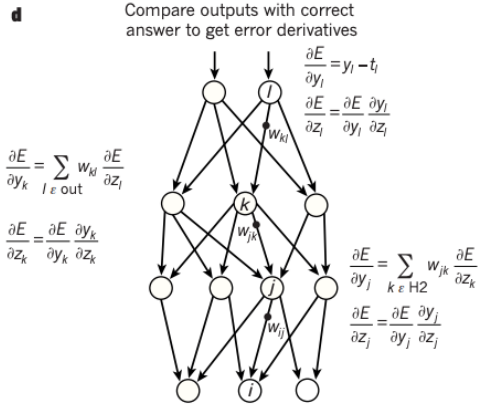
Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction:

$\hat{y} = w_1 m + w_1 n$



Forward pass



Back propagation

Toy Example of Gradient Descent in ML Models

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

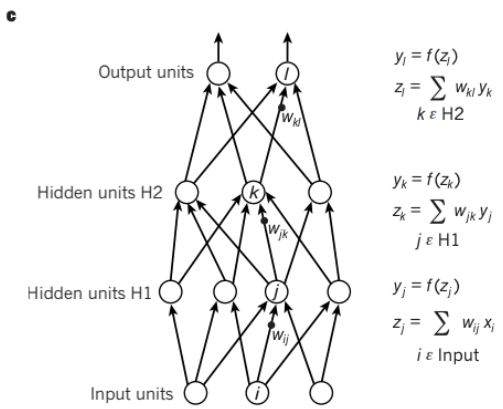
Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate: 0.1

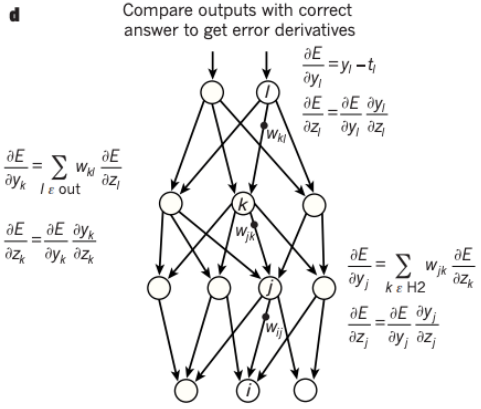
Initial weights: $w1 = 1, w2 = 1$

Step 2: Compute the Loss

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$$



Forward pass



Back propagation

Toy Example of Gradient Descent in ML Models

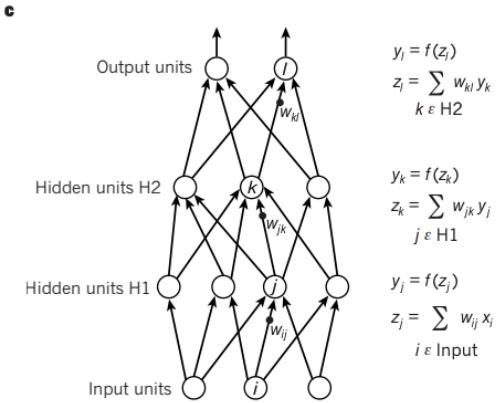
Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

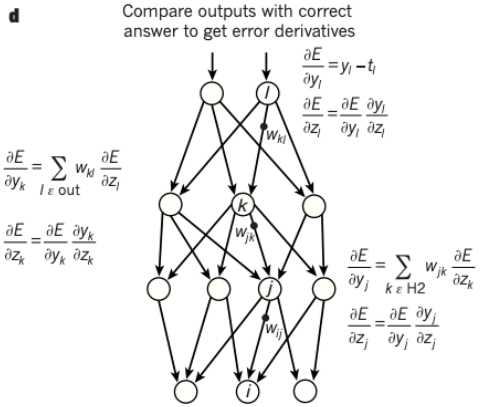
Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: w1 = 1, w2 = 1



Forward pass



Back propagation

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial (\hat{y} - y1)^2}{\partial w_1} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_1} = (w_1 m + w_2 n - y1)\, m$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial (\hat{y} - y1)^2}{\partial w_2} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_2} = (w_1 m + w_2 n - y1)\, n$$

Toy Example of Gradient Descent in ML Models

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

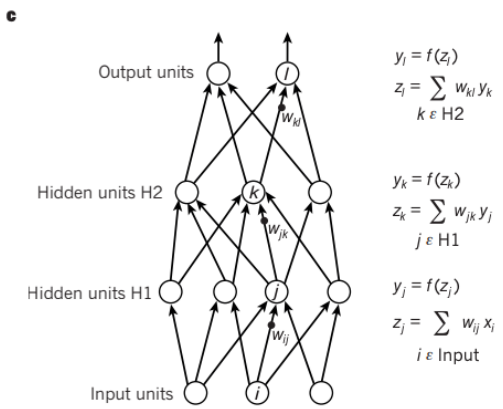
Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

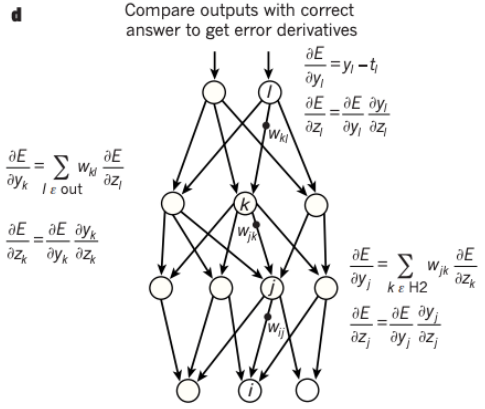
Initial weights: $w1 = 1, w2 = 1$

Step 4: Gradient descent update rule

$$w_i^{new} \leftarrow w_i^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$



Forward pass



Back propagation

$$w_1^{new} = w_1^{old} - 0.1 * (w_1 m + w_2 n - y1)\, m$$

$$w_2^{new} = w_2^{old} - 0.1 * (w_1 m + w_2 n - y1)\, n$$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: w1 = 1, w2 = 1

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial (\hat{y} - y1)^2}{\partial w_1} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_1} = (w_1 m + w_2 n - y1)\, m$$

$$= (1*2 + 1*1 - 4) * 2$$

$$= -2$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial (\hat{y} - y1)^2}{\partial w_2} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_2} = (w_1 m + w_2 n - y1)n$$

$$= (1 * 2 + 1*1 - 4) * 1$$

$$= -1$$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: w1 = 1, w2 = 1

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = -2 \qquad \frac{\partial \mathcal{L}}{\partial w_2} = -1$$

Step 4: Gradient descent update rule

$$w_i^{new} \leftarrow w_i^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$

$$w_1^{new} = w_1^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_1} = 1 - 0.1 * (-2) = 1.2 \qquad w_2^{new} = w_2^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_2} = 1 - 0.1 * (-1) = 1.1$$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = -2 \qquad \frac{\partial \mathcal{L}}{\partial w_2} = -1$$

Step 4: Gradient descent update rule

$$w_1^{new} = 1.2 \qquad w_2^{new} = 1.1$$

Does the updated weights help the model better fit the data?

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = (\hat{y} - y)^2$

Learning rate: 0.1

Initial weights: w1 = 1, w2 = 1

Previous Model: $\hat{y} = 1.0 * x_1 + 1.0\ x_2$

Previous Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

Current Model: $\hat{y} = 1.2 * x_1 + 1.1\ x_2$

Current Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 1/2 (1.2 * 2 + 1.1 * 1 -4)^2
= **0.125**

Does the updated weights help the model better fit the data?

Syllabus needs to be updated

We observe Spring break from 03/22/2026 – 03/30/2026.

SPRING SEMESTER 2026

Open Registration Ends	January 19, Monday
Martin Luther King, Jr. Recognition Holiday	January 19, Monday
Spring semester opens 8:00 a.m.	January 20, Tuesday
Classwork begins 8:00 a.m.	January 20, Tuesday
Spring Recess begins 8:00 a.m.	March 12, Thursday
Mid-Semester	March 14, Saturday
Spring Recess ends 8:00 a.m.	March 16, Monday
Spring Break begins 8:00 a.m.	March 22, Sunday
Spring Break ends 8:00 a.m.	March 30, Monday
Last Class Day	May 8, Friday

The corresponding content will be moved forward.